Advanced Topics of Programming Languages

1.1 - Memory Models

Example:

Thread A

```
void foo(void) {
  a = 1;
  b = 1;
}
```

Thread B

```
void bar(void) {
  while (b != 1) {};
  assert a==1;
}
```

↳ intuition: the assertion in bar should never fail.

But, given enough tries, it actually might fail!

↳ for example, some compilers may reorder the statements in foo since they don't depend on each other. Then b could be 1 but not a.

So, we need to define a memory model.

Memory interactions behave differently in the presence of

- multiple concurrent threads
- data replication in hierarchical and/or distributed memory systems
- deffered communication of updates

Memory models are a product of negotiating (basically compromising between these two)

- restrictions of freedom of implementation to guarantee race-related properties
- establishment of freedom of implementation to enable program and machine model optimisations

→ modern languages include the memory model in their language definition.

1.2 - Strict Consistency

Strict Consistency: independently of which process reads or writes, the value from the most recent write to a location is observable by reads from the respective location **immediately after the write** occurs.

↳ also, having an absolute global time is problematic

↳ although intuitively desired, it's **not physically possible!** Strict consistency is too strong to be realistic.

1.3 - Happened Before Relation

Example:

Thread A

```
void foo(void) {
    a = 1;
    b = 1;
}
```

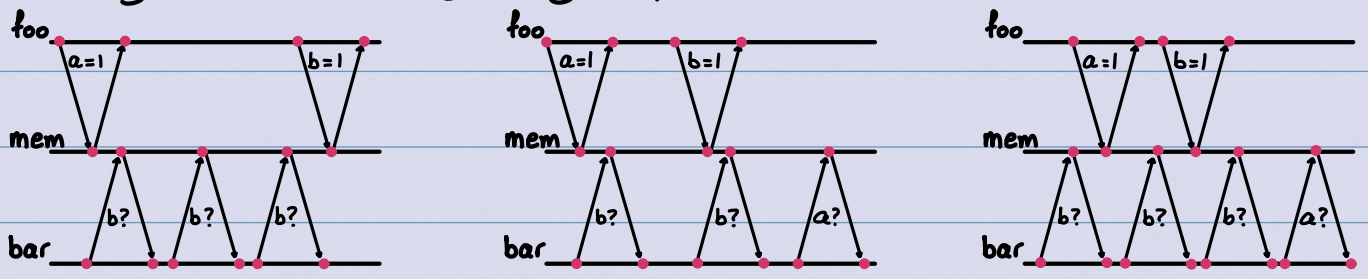
Thread B

```
void bar(void) {
    while (b != 1) {};
    assert a == 1;
}
```

- initial state of a and b is 0
- thread A stores to a before it stores to b
- thread B should see b go to 1 before executing the assert statement
- the assert statement should always hold

↳ here, correctness means: storing 1 to a **happens before** (relative-time!) loading 1 to b

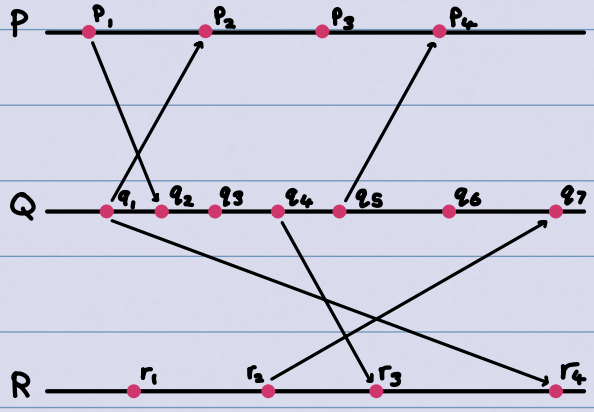
Still, any of the following may happen:



Events in a Distributed System

Given a distributed system of **processes** P, Q, R, ..., each process P consists of **events** P_1, P_2, ...

Example:



- event p_i in process P happened before event p_{i+1}
- if p_i is an event that sends a message to Q, then there is some event q_j in Q that receives this message, and p_i happened before q_j.

If an event p happened before an event q, then $p \to q$.

Observe:

- $\to$ is partial (neither $p \to q$ or $q \to p$ may hold)
- $\to$ is irreflexive ($p \to p$ never holds)
- $\to$ is transitive ($p \to q \wedge q \to r \Rightarrow p \to r$).
- $\to$ is asymmetric (if $p \to q$ then $\neg(q \to p)$).

$\rightsquigarrow$ the $\to$ is a strict partial order.

Also, let $p \not\to q$ abbreviate $\neg(p \to q)$.

1.4 - Logical Clocks

Two distinct events are said to be concurrent if $p \not\to q$ and $q \not\to p$.

↳ in the previous example, $p_1 \to r_4$ so they're not concurrent, but p_3 and q_3 are concurrent since $p_3 \not\to q_3$ and $q_3 \not\to p_3$.

Event Ordering: use a Logical Clock C, ie, C assigns a global timestamp $C(p)$ to each event p.

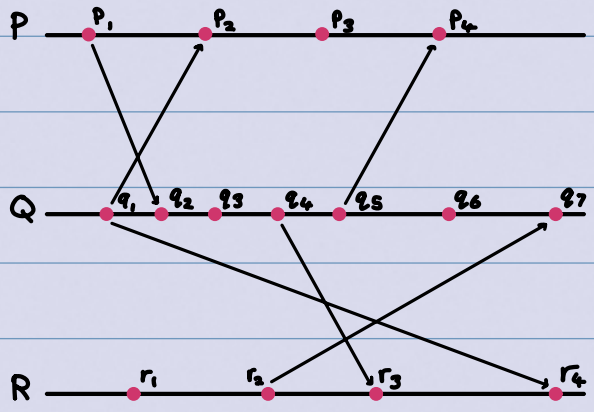
Function C satisfies the clock condition if, for any events p and q, $p \to q \Rightarrow C(p) < C(q)$.

For a distributed system, the clock condition holds if and only if:

1. p_i and p_j are events of P and $p_i \to p_j$ then $C(p_i) < C(p_j)$
2. p is the sending message by process P and q is the reciever of this message in process Q then $C(p) < C(q)$.

- a logical clock C with unique timestamps that satisfies the clock condition describes a total order $a < b$ (with $C(a) < C(b)$) that embeds the strict partial order $\to$
- the set defined by all those C is exactly the set of sequential executions possible to observe in the system.

Example: defining C satisfying the Clock Condition



One possible solution:

e	p_1	p_2	p_3	p_4
C(e)	1	4	7	12

e	q_1	q_2	q_3	q_4	q_5	q_6	q_7
C(e)	2	3	5	6	11	13	14

e	r_1	r_2	r_3	r_4
C(e)	8	9	10	15

1.5 - Cache Coherence

Idea: use happened-before diagrams to model more relaxed memory models

Given a path through each of the threads of a program:

- consider the actions of each thread as events of a process
- use more processes to model more memory

$\rightsquigarrow$ concisely represent some interleavings $\Rightarrow$ we establish models for memory coherence

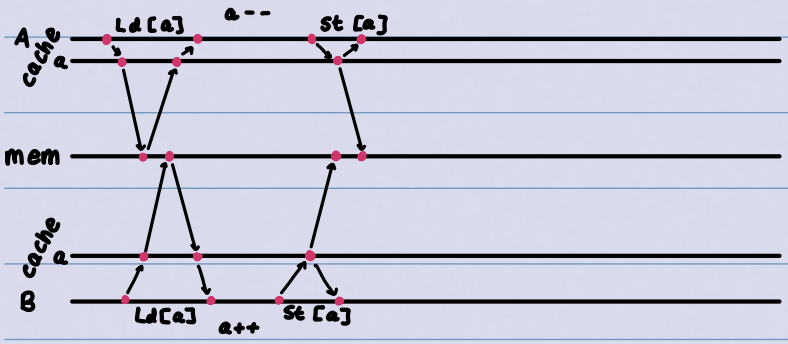
Weakening the Model: Introducing Caches

Observation: programs often access the same memory area repeatedly

Idea: processors use quickly accessible caches to avoid a costly round-trip to RAM for each memory access.

Model: each cache line as one process

Example: program that decrements a



issue here: process B has no effect here since the replicated a in the cache from process A overwrote process B's changes!

Cache Coherence:

1. Memory operations in program order ($\leq$) are embedded into the memory order ($\sqsubseteq$).

$$Op_i[a] \leq Op_i[a]' \Rightarrow Op_i[a] \sqsubseteq Op_i[a]'$$

2. For each address a, memory order $\sqsubseteq$ is a total order on all accesses to a
3. A load's value is determined by the latest store with respect to memory order: $val(Ld_i[a]) = val(St_j[a] \mid St_j[a] = \max_{\sqsubseteq}(\{St_k[a] \mid St_k[a] \sqsubseteq Ld_i[a]\})$

With:

- $Op_i[a]$: any memory access to address a by CPU i
- $Ld_i[a]$: a load from address a by CPU i
- $St_i[a]$: a store from address a by CPU i
- Program order $\leq$ being specified by the control flow of the program for each CPU

1.6 - Sequential Consistency

Messaging Protocol Example:

Thread A

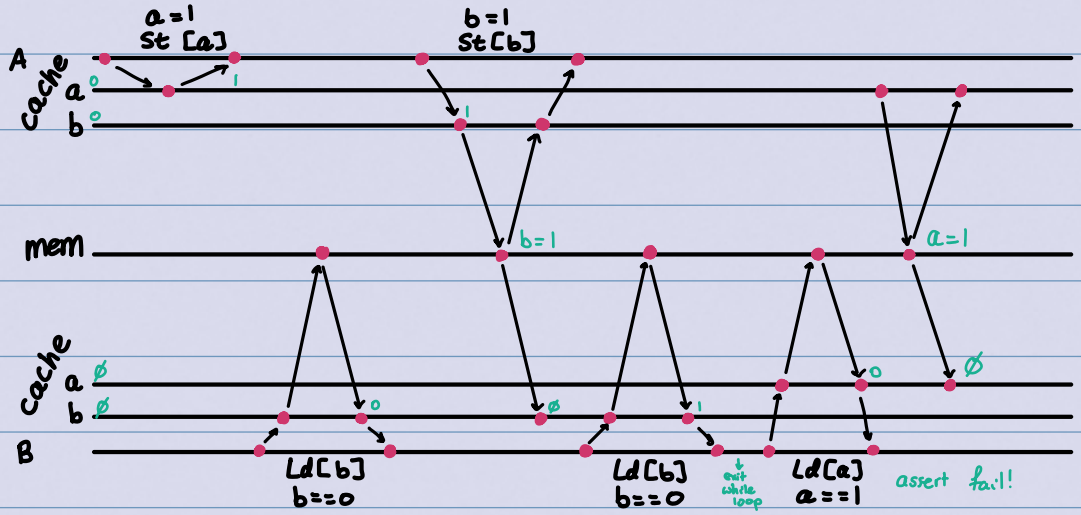
```
void foo(void) {
    a = 1;
    b = 1;
}
```

Thread B

```
void bar(void) {
    while (b != 1) {};
    assert a == 1;
}
```

- in all examples, the initial value of variables is assumed to be 0
- Suppose that a and b reside in different cache line
- assume that a cache line is larger than the variable itself

initially: each cache line one process, A caches a=0, b=0, and B's cache is empty



In this example, we can see a coherent cache but sequential inconsistency!

Sequential Consistency Conditions: the **result of any execution** is the **same as if the memory operations**

1. of **all processors joined** were executed in **some sequential order**
2. and the **operations of each individual processor appear in this sequence** in the order specified by its program

Idea for **showing that a system is NOT sequentially consistent**:

- **pick a result** obtained from a program run on a sequentially consistent system

- pick an execution ② and a total ordering of all operations ①
- add extra processes to model other system components
- show that a different total order $\sqsubseteq'$ that embeds $\leq$ exists, for which the result differs.

1.7 - Sequential Consistency (Formal Definition)

Sequential Consistency:

1. Memory operations in program order ($\leq$) are embedded into the total memory order ($\sqsubseteq$)

$$Op_i[a] \leq Op_i[b]' \Rightarrow Op_i[a] \sqsubseteq Op_i[b]'$$

2. A load's value is determined by the latest store wrt memory order

$$val(Ld_i[a]) = val(St_j[a] \mid St_j[a] = \max_{\sqsubseteq}(\{St_k[a] \mid St_k[a] \sqsubseteq Ld_i[a]\})$$

↳ this definition looks close to the definition of basic coherence! the difference is that the embedding part covers all memory locations instead of particular ones!

→ for sequential consistency, caches and memory need to collaborate in a Cache Coherence Protocol.

Benefits of Sequential Consistency

- concisely represent all interleavings that are due to variations in timing
- synchronisation using time is uncommon for software
- a good model for correct behaviors of concurrent programs

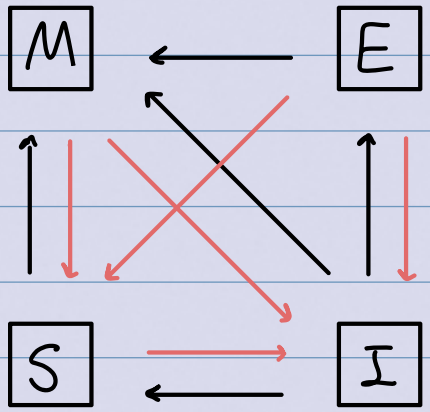
↳ suitable for concurrent processors that acquire exclusive access to memory

1.8 - Introduction to MESI Protocol

Problem: Keeping track of the latest copy of a value in distributed

local mirror images

Idea: attach a state machine to each cached line



Each cache line is in one of the states:

I: it's invalid and ready for reuse

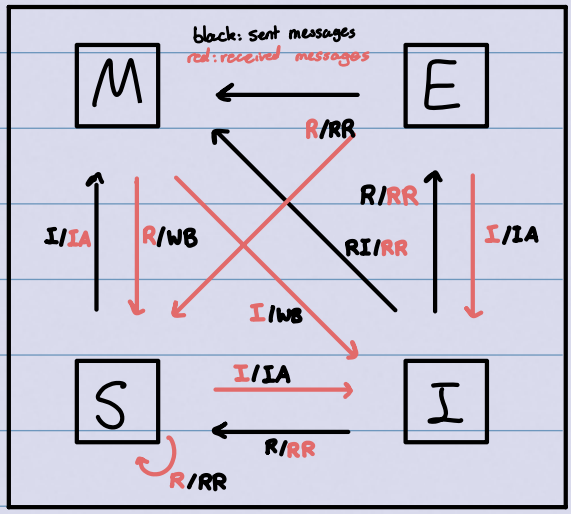
S: other caches have an identical copy; it's shared

E: the content is in no other cache; it's exclusive

M: the content is exclusive to this cache and has furthermore been modified

↳ the global state of cache lines is kept consistent by sending messages

Moving data between caches is coordinated by sending messages



R: Read → sent if CPU needs to load from an address

RR: Read Response → when in state E or S, reply to R, carries the data for the requested address

I: Invalidate → ask others to evict a cache line

IA: Invalidate Acknowledged → reply to I indicating that a cache line was evicted

RI: Read Invalidate → like Read + Invalidate (also called "read with intent to modify")

WB: Writeback → Special RR – when in state M, reply to R and as a side effect notifying main memory about modifications to the cache line

we mostly consider messages between processors. Upon read invalidate, a processor replies with RR / WB before the IA is sent.

1.9 – MESI Example

we'll write the content of a cache line as:

- M_x: modified, with value x
- E_x: Exclusive, with value x
- S_x: shared, with value x
- I: Invalid

Example:

Thread A

```
void foo(void) {
    a = 1;      // A.1
    b = 1;      // A.2
}
```

Thread B

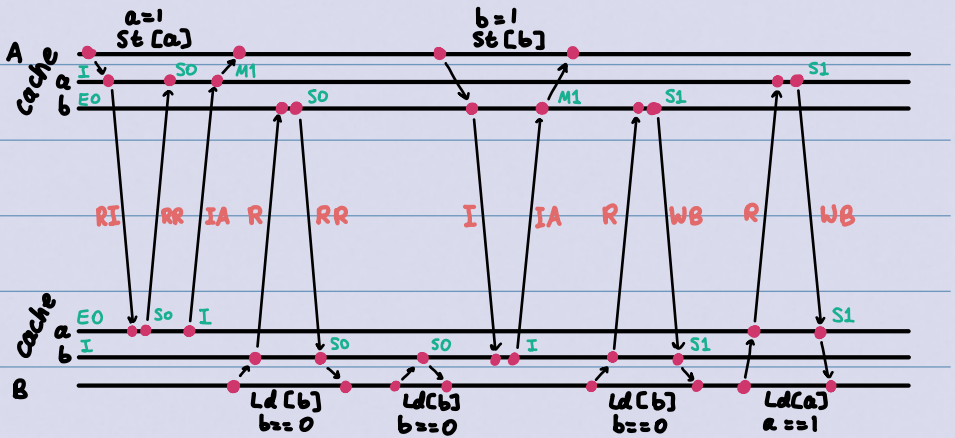
```
void bar(void) {
    while (b != 1) {};   // B.1
    assert a == 1;       // B.
}
```

Statement	CPU A		CPU B		RAM		message
	a	b	a	b	a	b	
A.1	I	I	I	I	0	0	) read invalidate of a from CPU A
	I	I	I	I	0	0	) invalidate ack. of a from CPU B
	I	I	I	I	0	0	) read response of a=0 from RAM
B.1	M1	I	I	I	0	0	) read of b from CPU B
	M1	I	I	I	0	0	) read response with b=0 from RAM
B.1	M1	I	I	E0	0	0	
A.2	M1	I	I	E0	0	0	
	M1	I	I	E0	0	0	) read invalidate of b from CPU A
	M1	S0	I	S0	0	0	) read response of b=0 from CPU B
B.1	M1	M1	I	I	0	0	) invalidate ack. of b from CPU B
	M1	M1	I	I	0	0	) read of b from CPU B
B.2	M1	S1	I	S1	0	1	) write back of b=1 from CPU A
	M1	S1	I	S1	0	1	) read of a from CPU B
	S1	S1	S1	S1	1	1	) write back of a=1 from CPU A

1.10 - MESI in the context of happened-before

Example:

- each cache line one process
- A caches b=0 as E
- B caches a=0 as E



Observations:

- each memory access must complete before executing next instruction ⇝ add edge
- second execution of test b==0 stays within cache ⇝ no traffic to mem / bus!!

Chapter 1 Summary: MESI Cache Coherence Protocol

Cache Coherence:

- A cache coherent system must appear to execute all threads' loads and stores to a single memory location in a total order that respects the program order of each thread
  ↳ memory order can still be partial!
- MESI cache coherence protocol ensures cache coherence for processors with caches

Sequential Consistency

- specifies that the system must appear to execute all threads' loads and stores to all memory locations in a total order that respects the program order of each thread
- a characterisation of well-behaved programs
- a model for differing speed of execution
- for fixed paths through the threads and a total order between accesses to the same variables: executions can be illustrated by a happened-before diagram with one process per cached variable
- MESI cache coherence protocol ensures sequential consistency for processors with simple caches.